

ANN-Cache: Accelerating Approximate Nearest Neighbor Search via Caching

Lingli Li , Zhanyu He, Zhuo Zhang , and Ying Deng 

Abstract—Approximate nearest neighbor (ANN) search in the high-dimensional Euclidean space is a pivotal problem for various data science and AI applications. The performance of traditional in-memory ANN methods decreases dramatically when handling large-scale data due to their substantial expansion of main memory. Recently, disk-resident ANN methods have drawn considerable attention to enable large-scale ANN search. However, despite their significant improvements, they still suffer from high query latency due to heavy I/O cost. In this paper, we propose ANN-CACHE, an I/O-efficient cache-conscious ANN framework that leverages the skew in query workloads to reduce I/O cost. To the best of our knowledge, we are the first to formally study exploring caching to accelerate disk-based ANN search. ANN-CACHE consists of two components: i) a discriminator that checks whether a query's ANN points are cached; and ii) an LSH-based cache that stores historical queries and their results to support efficient ANN search. Experimental results indicate that ANN-CACHE achieves $3\times-9\times$ speedup over baselines while maintaining comparable or better accuracy, incurring relatively low extra memory overhead, and demonstrating strong adaptivity to dynamic workloads.

Index Terms—High-dimensional vector, approximate nearest neighbor search, caching, disk-based index.

I. INTRODUCTION

NOWADAYS, ANN search is one of the most challenging and critical problems in a wide range of diverse application domains, including pattern recognition [1], information retrieval [2], data mining [3], and machine learning [4]. Over the last few decades, various index structures and search algorithms for ANN search have been proposed (see the latest survey [5] for reference), with the majority of these approaches being memory-resident [6], [7], [8]. They assume that both the dataset and its corresponding index can be accommodated in the main memory (DRAM), which can hardly meet the storage requirements for large-scale datasets [9]. To enable large-scale ANN search, a few studies [9], [10], [11] have recently explored efficient disk-resident indexes. These methods aim to achieve

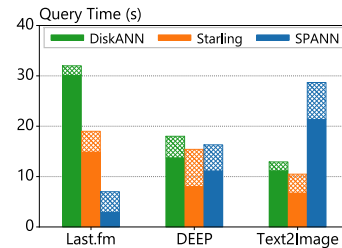


Fig. 1. Breakdown of performance.

high recall and low latency by minimizing I/O cost while utilizing small amounts of memory resources. However, despite their significant improvements, they still suffer from high query latency due to substantial I/O cost.

To illustrate, we provide a breakdown of the query execution times of three disk-resident methods, DISKANN [9], STARLING [11] and SPANN [10] on three real-world datasets, Last.fm, DEEP, and Text2Image when targeting 0.95 recall with $k = 10$. As Fig. 1 shows, the total query execution time comprises the costs for: i) in-memory computation (hatched bars) which includes the index lookup and distance computation; and ii) I/O cost (solid bars) that loads the relevant index and data points from disk into memory. The results show that I/O cost is the dominant factor affecting performance. For example, I/O accounts for 76% - 94% of the total time for DISKANN, 52% - 80% for STARLING, and 41% - 69% for SPANN. This indicates that *reducing I/O operations holds significant potential for improving query efficiency*.

A natural idea is to apply caching to reduce I/O cost, which is regarded as the most effective technique for data access acceleration [12], [13]. However, directly caching historical queries can hardly accelerate ANN search, because the likelihood of encountering identical queries is quite low in real workloads, rendering an extremely low cache hit rate. For example, consider an image or document dataset where vectors represent embeddings of images or documents. It is unlikely to find two identical image or document queries, which leads to a low hit rate. Fortunately, we observe that caching can be useful when we loosen the condition from retrieving identical queries to retrieving similar queries in skewed query workload scenarios, which are common in real applications [14]. Specific observations are illustrated in the following.

Observation 1: *Many real-world workloads in ANN applications are highly skewed.* For example, in music recommendation

Received 19 July 2025; revised 13 March 2026; accepted 2 June 2026. Date of publication 5 June 2026; date of current version 8 August 2026. This work was supported by the National Natural Science Foundation of China under Grant 62472148 and in part by the Natural Science Foundation of Heilongjiang Province in China under Grant PL2024F028. Recommended for acceptance by K. Zheng. (Corresponding author: Lingli Li.)

The authors are with the Department of Computer Science, Technology, Heilongjiang University, Heilongjiang 150080, China (e-mail: lilingli@hlju.edu.cn; hezhanyu2001@163.com; zhang_zhuo2022@163.com; 20230594@s.hlju.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TKDE.2026.3700562>, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2026.3700562

scenarios, Last.fm [15] provides a highly skewed ANN query set, where approximately 90% of the total queries access only 10% of total vectors. Similarly, in retrieval-augmented generation (RAG) systems, it has been reported that only 5% of vectors are accessed by 40%-60% of the ANN queries [16]. Furthermore, [17] claims that ANN queries in well known open-source vector search datasets can exhibit highly skewed access patterns, where certain regions are significantly more important than others. These highly skewed query workloads provide strong justification for caching hot data in main memory.

Observation 2: Similar queries often access similar data. This is because a neighbor of a neighbor is also likely to be a neighbor [18]. In other words, given a query q , rather than searching for an identical historical query with a very small chance, it is more practical to find historical queries similar to q , whose nearest neighbors are also likely to be the nearest neighbors of q . Thus, the issue of low hit rate can be effectively mitigated by loosening the cache hit condition from *retrieving identical queries* to *retrieving similar queries*.

The above characteristics give us a chance to design an effective caching mechanism for ANN search by utilizing the knowledge of historical queries. This leads to two major challenges to be resolved.

Challenge 1: How to identify a cache hit? It is non-trivial to check whether the nearest neighbors of a given query are cached, without traversing the entire dataset. To this end, we transform this hard question into an easy one: given a query q , does there exist a historical query q' in the cache similar to q (denoted as $q' \approx q$)? This problem is analogous to the classical “membership testing” problem, i.e., given an element x , whether there exists an element y in a set S such that $x = y$. The membership testing problem can be efficiently solved by Bloom Filter [19]. The major difference between these two problems lies in the matching criterion: we relax the constraint from exact match ($=$) to approximate match ($\approx$). Therefore, we refer to our problem as “matching membership testing” (MMT). One way is to apply ANN methods to solve MMT, but it is too time consuming due to the heavy cost of distance computations. Thus, the first challenge is to design a light-weight discriminator for MMT that is both space- and time-efficient.

Challenge 2: How to design a cache data structure to support efficient ANN search? An intuitive idea is to cache each incoming query along with its ANN and construct an index over these cached queries. However, this approach has two major drawbacks. First, it exhibits high redundancy, since hot data points are shared by similar queries as their common nearest neighbors, resulting in multiple copies. Second, when the cache is full, since most queries are unique, conventional eviction strategies become ineffective. This necessitates a cache design that stores both queries and their results with lower redundancy, with an effective eviction rule to delete stale queries and data points from the cache.

By addressing the aforementioned challenges, we propose a general caching framework for disk-based ANN methods, named ANN-CACHE. To the best of our knowledge, this is the first work to formally study exploring caching to accelerate

disk-based ANN search. To address the first challenge, we design a novel data structure for the discriminator, which combines both Bloom Filter and Locality Sensitive Hashing (LSH). It is able to support efficient MMT as it combines the advantages of both Bloom Filter and LSH, i.e., supporting compact “membership testing” and “similarity search”. To address the second challenge, we propose an LSH-based data structure to store historical queries and their ANNs. By treating queries as keys and their ANNs as values, the cache enables constant-time lookup of similar queries’ results. Moreover, the cache reduces redundancy, as similar queries that collide usually share common nearest neighbors. By storing the union of results from collided queries, each data point only needs to be stored once.

Our main contributions are summarized as follows.

- We design a light-weight discriminator. By utilizing both Bloom Filter and LSH techniques, the discriminator supports efficient MMT in approximately constant time by incurring a small amount of memory overhead. In addition, the error rate can also be controlled by tunable parameters. Finally, the discriminator enables fast dynamic updates to be in accordance with the cache.
- We propose an LSH-based cache structure to support efficient ANN search. By utilizing historical queries and their ANNs, our method avoids unnecessary lookup in the disk-resident index. Furthermore, we introduce a pruning strategy that removes unpromising data points by computing projected distances via LSH functions. Finally, to adapt to workload shifts, we develop the eviction rule and cache update mechanism.
- We conduct extensive experiments on six realworld datasets. Experimental results demonstrate that our method consistently achieves high accuracy and gains $3\text{--}9\times$ speedup compared to the baselines, while incurring relatively low extra memory overhead and demonstrating strong adaptivity to dynamic workloads.

The rest of this paper is organized as follows. In Section II, we discuss preliminaries. Section III presents a brief overview. Sections IV and V detail the implementation of the proposed caching framework. We report the results of our experiments in Section VI. In Section VII, we review previous work. Finally, we conclude our work in Section VIII.

II. PRELIMINARIES

In this section, we introduce the problem definition in Section II-A and briefly review the LSH and the Bloom Filter in Sections II-B and II-C, respectively.

A. Problem Definition

Given a dataset $\mathcal{D} = \{o_i \in \mathbb{R}^d\}_{i=1,\dots,n}$ consisting of n data points in a d -dimensional Euclidean space, a query $q \in \mathbb{R}^d$, and a positive integer k , the target of exact kNN problem is to retrieve the k nearest points to q in $\mathcal{D}$ based on the Euclidean distance metric. The goal of ANN is to retrieve k neighbors that are close to being optimal. This paper focuses on the ANN problem. Unless otherwise specified, we use the terms ANN and kNN interchangeably.

We use *recall* to measure the accuracy of ANN result. Let $\mathcal{R}(q) = \{o_1, \dots, o_k\}$ denote the ANN result of q , and $\mathcal{R}^*(q) = \{o_1^*, \dots, o_k^*\}$ denote the exact kNN of q , then the recall of $\mathcal{R}(q)$ is defined as $recall = \frac{|\mathcal{R}(q) \cap \mathcal{R}^*(q)|}{k}$.

B. Locality-Sensitive Hashing

A family of LSH functions is able to project closer data points into the same bucket with higher probability. Let the ball of radius r centered at a data object $o \in \mathcal{D}$ be defined as $B(o, r) = \{q \in \mathbb{R}^d \mid \|o, q\| \leq r\}$. Formally, an LSH function family in Euclidean space is given below [20].

Definition 1: (LSH) An LSH function family $\mathcal{H} = \{h : \mathbb{R}^d \rightarrow U\}$ is called (r, cr, p_1, p_2) -sensitive if for any $o, q \in \mathbb{R}^d$:

- if $o \in B(q, r)$, then $\Pr_{\mathcal{H}}[h(o) = h(q)] \geq p_1$;
- if $o \notin B(q, cr)$, then $\Pr_{\mathcal{H}}[h(o) = h(q)] \leq p_2$,

where $c > 1$ and $p_1 > p_2$.

Two typical LSH families for Euclidean space are formally defined as follows [21], [22]:

$$h(o) = \left\lfloor \frac{\vec{a} \cdot \vec{o} + b}{w} \right\rfloor, \quad (1)$$

$$h'(o) = \lfloor \vec{a} \cdot \vec{o} \rfloor, \quad (2)$$

where $\vec{o}$ is the vector representing point $o \in \mathbb{R}^d$, $\vec{a}$ is a d -dimensional random vector with each entry being independently drawn from the standard normal distribution $\mathcal{N}(0, 1)$, w is a pre-specified bucket width, and b is a real number uniformly randomly sampled from the interval $[0, w)$. Note that, the function $h(o)$ (1) that projects nearby points into the same bucket with high probability is used for retrieving candidate points of kNN; while the function $h'(o)$ (2) that preserves the similarity between points in the projected space is employed to prune unpromising data points by reducing distance computation costs.

C. Bloom Filter

Bloom Filter [19] is a probabilistic data structure designed to represent a set $S = \{x_1, \dots, x_n\}$ of n points using a bit array of length m . Initially, all bits in the array are set to 0. It employs l independent hash functions h_1 - h_l , to map each point in S to l positions in the bit array. To insert a point x into the filter, each hash function $h_i(x)$ (for $i = 1, \dots, l$) maps x to a position in the range $[0, m - 1]$, and the corresponding bits are set to 1. To check whether a point q belongs to S , one verifies whether all l hash functions map q to positions in the bit array that are set to 1. If this condition holds, q is considered a member of S ; otherwise, q is not in S . Such a construction and query mechanism ensure that there are no false negatives (members taken as nonmembers) but possibly false positives (nonmembers taken as members). Specifically, the false positive rate (FPR) can be calculated as: $FPR = (1 - (1 - \frac{1}{m})^{nl})^l$.

III. ANN-CACHE OVERVIEW

To address the I/O bottleneck in disk-based ANN search, we propose ANN-CACHE, a caching framework (Fig. 2) that aims to reduce unnecessary disk accesses. ANN-CACHE takes a historical query set as input and builds two key components:

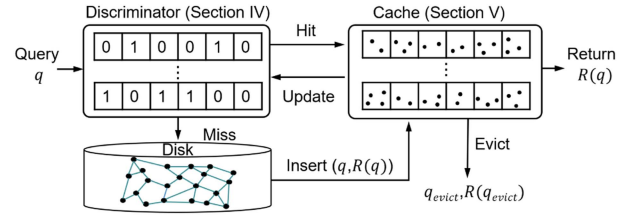


Fig. 2. Overview of the ANN-CACHE framework.

(i) a discriminator (Section IV) that predicts cache hits for incoming queries; and (ii) a cache (Section V) storing historical queries along with their kNNs (likely to be hot data). These two components enable us to utilize the knowledge of historical queries to accelerate search on the one hand, and update in a low cost to maintain high performance under workload shifts on the other hand.

The Discriminator $\mathcal{D}_Q$: Given a query q , if the discriminator $\mathcal{D}_Q$ returns `true`, it indicates a *cache hit*, allowing results to be retrieved directly from the cache. Otherwise, it indicates a *cache miss*, which triggers fetching results from the disk-resident index. When the cache is updated, i.e., new elements are inserted or stale elements are removed, the discriminator $\mathcal{D}_Q$ should be updated accordingly to maintain consistency with the cache. To enable efficient MMT, we design a hybrid discriminator combining Bloom Filters and LSH, which achieves constant-time lookup complexity and consumes a small amount of memory.

The Cache $\mathcal{C}_Q$: Given a query q , the cache $\mathcal{C}_Q$ retrieves the historical queries that are similar to q and collect their kNNs as candidate results. To enable fast similarity search on both historical queries and their kNNs, we employ multiple LSH-based hash tables. Further, we utilize a pruning strategy to optimize the query efficiency. It uses the estimated distance to filter out irrelevant candidates. When workload shifts, some cached data would become cold that should be removed and new data would become hot that should be added. To accommodate workload change, we propose the update algorithm of the cache. Further, to maintain consistency between queries and their associated ANN points, we develop an eviction rule that deletes data points and queries collaboratively.

IV. THE DISCRIMINATOR

This section details the design of the discriminator; its overall workflow is depicted in Fig. 3. We first present the construction phase (Section IV-A), then describe the lookup algorithm with theoretical analysis (Section IV-B), and finally introduce the update operations (Section IV-C).

A. Construction

Given a cached historical query set $\mathcal{Q}$, a discriminator $\mathcal{D}_Q$ of $\mathcal{Q}$ is used to answer the MMT problem. Let $\approx$ represent a threshold-based match constraint that determines if two queries are close enough to be considered “matched”, i.e., $a \approx b$ iff the

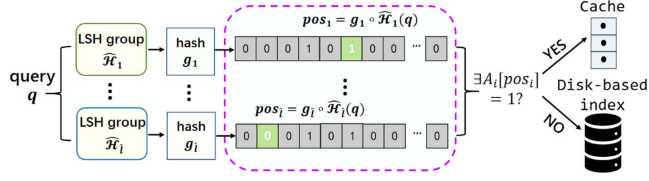


Fig. 3. Discriminator lookup workflow. The discriminator consists of $\hat{l}$ bit-arrays $A_1, \dots, A_{\hat{l}}$. For an incoming query q , it checks each array A_i at the position $g_i \circ \hat{\mathcal{H}}_i(q)$. If one of the bits is set to 1, a cache hit occurs and triggers a cache lookup; otherwise, a cache miss is signaled, and a disk-based ANN search is executed.

Euclidean distance $\text{dis}(a, b) \leq \epsilon$. The formal definition of MMT is presented as follows.

Problem 1. (MMT Problem): Given a query point q and a cached query set $\mathcal{Q} = \{q'_1, \dots, q'_s\}$ of size s , the MMT problem is to determine whether there exists a query $q' \in \mathcal{Q}$ that matches q ($q' \approx q$). If so, we say that q is hit by $\mathcal{Q}$ (cache hit), denoted by $q \in \mathcal{Q}$; otherwise, q is missed by $\mathcal{Q}$ (cache miss), denoted as $q \notin \mathcal{Q}$.

On the one hand, Bloom Filter, the commonly used data structure for the membership problem, cannot be directly applied to answer the MMT problem, since hash functions do not support approximate match constraint. On the other hand, LSH, the classical technique for approximate matching, cannot be directly applied to answer the MMT problem efficiently either, because performing the test is space-expensive. It requires $\mathcal{O}(\hat{r}\hat{l}s)$ space to store $\hat{l}$ copies of s $\hat{r}$ -dimensional projected points, where $\hat{r}$ is the dimension of each projected space, and $\hat{l}$ is the number of independent projected spaces. To address these issues, we design a novel data structure that combines the advantages of Bloom Filter and LSH by constructing multiple bit arrays based on LSH functions.

Data Structure: The discriminator of $\mathcal{Q}$, denoted as $\mathcal{D}_{\mathcal{Q}}(\hat{m}, \hat{l}, \hat{r})$, is composed of $\hat{l}$ bit-arrays $A_1, \dots, A_{\hat{l}}$, each of size $\hat{m}$. Each array A_i is associated with a set of $\hat{r}$ independent LSH functions based on Equation 1, denoted as $\hat{\mathcal{H}}_i = \{h_i^1, \dots, h_i^{\hat{r}}\}$. We give the pseudo-code of discriminator construction in Algorithm 1. Initially, all bits in A_i are initialized to 0 (line 1). For each d -dimensional point q' in $\mathcal{Q}$ (line 4), q' is mapped to an $\hat{r}$ -dimensional space by $\hat{\mathcal{H}}_i$ for each array A_i , denoted as $\hat{\mathcal{H}}_i(q') \leftarrow [h_i^1(q'), \dots, h_i^{\hat{r}}(q')]$. Then $\hat{\mathcal{H}}_i(q')$ is assigned to a bit at location $g_i \circ \hat{\mathcal{H}}_i(q')$ in A_i , and the corresponding bit is set to 1 (line 6), where $g_1, \dots, g_{\hat{l}}$ are independent hash functions with range $[0, \hat{m} - 1]$. Note that, by setting $\hat{m}$ to be sufficiently large, we can reasonably assume that two points are hashed to the same position on A_i iff they have identical values on $\hat{\mathcal{H}}_i$. Clearly, Algorithm 1 requires $\mathcal{O}(d\hat{r}\hat{l}s)$ time to construct and consumes $\mathcal{O}(\hat{m}\hat{l})$ bits to store s points.

Example 1: A toy example of the discriminator construction is shown in Fig. 4. Let $\mathcal{Q} = \{q'_1, q'_2\}$, $\hat{m} = 8$, and $\hat{l} = 2$. Initially, all bits in both arrays are set to 0. Then, for each query, two hash values are calculated. Suppose that $g_1 \circ \hat{\mathcal{H}}_1(q'_1) = 2$, $g_2 \circ \hat{\mathcal{H}}_2(q'_1) = 1$, $g_1 \circ \hat{\mathcal{H}}_1(q'_2) = 2$, and $g_2 \circ \hat{\mathcal{H}}_2(q'_2) = 3$. Therefore, the bit at position 2 is set to 1 in the first array A_1

Algorithm 1: Build-Discriminator ($\mathcal{Q}, \hat{m}, \hat{l}, \hat{r}$).

Input: a query set $\mathcal{Q}$ and parameters $\hat{m}, \hat{l}, \hat{r}$

Output: the discriminator $\mathcal{D}_{\mathcal{Q}}(\hat{m}, \hat{l}, \hat{r})$

- 1 Initialize $\hat{l}$ arrays: $A_1, \dots, A_{\hat{l}}$ with all bits set to 0;
 - 2 $\mathcal{D}_{\mathcal{Q}} \leftarrow \{A_1, \dots, A_{\hat{l}}\}$;
 - 3 Initialize $\hat{l}$ sets of $\hat{r}$ independent LSH functions $\hat{\mathcal{H}}_1, \dots, \hat{\mathcal{H}}_{\hat{l}}$ and $\hat{l}$ independent hash functions $g_1, \dots, g_{\hat{l}}$;
 - 4 **for each** $q' \in \mathcal{Q}$ **do**
 - 5 **for** $i \leftarrow 1$ **to** $\hat{l}$ **do**
 - 6 $A_i[g_i \circ \hat{\mathcal{H}}_i(q')] \leftarrow 1$;
 - 7 **return** $\mathcal{D}_{\mathcal{Q}}(\hat{m}, \hat{l}, \hat{r})$;
-

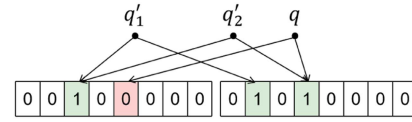


Fig. 4. An example of the discriminator.

Algorithm 2: Discriminator-Lookup (q).

Input: a query q

Output: true or false

// check if it is a cache hit

- 1 **for** $i \leftarrow 1$ **to** $\hat{l}$ **do**
 - 2 // compute the position of q in A_i
 - 3 $pos \leftarrow g_i \circ \hat{\mathcal{H}}_i(q)$;
 - 4 **if** $A_i[pos] == 1$ **then**
 - 5 **return** true;
 - 5 **return** false;
-

(left), and the bits at positions 1 and 3 are set to 1 in the second array A_2 (right).

B. Lookup

The pseudo-code of discriminator lookup is presented in Algorithm 2. Given a query q , to answer the MMT problem, $\mathcal{D}_{\mathcal{Q}}$ checks whether the position $g_i \circ \hat{\mathcal{H}}_i(q)$ of some A_i is set to 1. If $\exists i \in [1, \hat{l}]$, s.t., $A_i[g_i \circ \hat{\mathcal{H}}_i(q)] = 1$, then $\mathcal{D}_{\mathcal{Q}}$ returns true, indicating that q is hit by cache. Otherwise, $\forall i \in [1, \hat{l}]$ s.t., $A_i[g_i \circ \hat{\mathcal{H}}_i(q)] = 0$, then $\mathcal{D}_{\mathcal{Q}}$ returns false, indicating that q is missed by cache. Clearly, the time complexity of Algorithm 2 is $\mathcal{O}(d\hat{r}\hat{l})$.

Example 2: Consider a query q as illustrated in Fig. 4. The position of q in the first array is computed, say $g_1 \circ \hat{\mathcal{H}}_1(q) = 4$. Since the bit at this position is 0, we proceed to compute the position of q in the second array, which is $g_2 \circ \hat{\mathcal{H}}_2(q) = 3$. Since the bit at this position is 1, $\mathcal{D}_{\mathcal{Q}}$ returns true, indicating a cache hit.

Theoretical Analysis: Due to the use of LSH functions, $\mathcal{D}_{\mathcal{Q}}$ yields both false positives ($q \notin \mathcal{Q}$ but $\mathcal{D}_{\mathcal{Q}}$ returns true) and false negatives ($q \in \mathcal{Q}$ but $\mathcal{D}_{\mathcal{Q}}$ returns false). However, the error rate

Algorithm 3: Discriminator-Insert (q_{new}).

Input: a query q_{new}
1 for $i \leftarrow 1$ **to** $\hat{l}$ **do**
2 $A_i[g_i \circ \hat{\mathcal{H}}_i(q_{new})] \leftarrow 1;$

of $\mathcal{D}_Q$ can be controlled by properly setting the parameters $\hat{r}$ and $\hat{l}$. We begin our theoretical analysis by deriving the collision probability for matching and non-matching point pairs. Then, we show how this probability translates into the overall error rate of $\mathcal{D}_Q$.

Let $\varphi(x)$ denote the probability density function (PDF) of the standard normal distribution $\mathcal{N}(0, 1)$, and let $\tau = \text{dis}(q, q')$ represent the distance between q and q' . According to [21], the collision probability between q and q' under a single LSH function is given by:

$$p(\tau) = \Pr[h(q) = h(q')] = \int_0^w \frac{1}{\tau} \cdot \varphi\left(\frac{t}{\tau}\right) \cdot \left(1 - \frac{t}{w}\right) dt.$$

Let $\varphi(x; \mu, \sigma^2)$ denote the PDF of the normal distribution with mean μ and variance σ^2 . We assume that the distance between matching point pairs follows $\mathcal{N}(\epsilon_1, \sigma_1^2)$, while the distance between non-matching points follows $\mathcal{N}(\epsilon_2, \sigma_2^2)$. The expected collision probability p_1 (resp. p_2) for matching (resp. non-matching) point pairs can then be computed as:

$$p_1 = \int_0^\epsilon \varphi(\tau; \epsilon_1, \sigma_1^2) p(\tau) d\tau, \quad (3)$$

$$p_2 = \int_\epsilon^\infty \varphi(\tau; \epsilon_2, \sigma_2^2) p(\tau) d\tau. \quad (4)$$

According to the above equations, we can derive the expected error rate of the discriminator as below.

Algorithm 4: Discriminator-Delete (q_{evict}).

Input: a query q_{evict}
1 for $i \leftarrow 1$ **to** $\hat{l}$ **do**
2 $A_i[g_i \circ \hat{\mathcal{H}}_i(q_{evict})] \leftarrow 0;$

Theorem 1: The expected error rate of the discriminator $\mathcal{D}_Q(\hat{m}, \hat{l}, \hat{r})$ is the sum of the expected FNR and expected FPR. Formally, we have:

$$\begin{aligned} \mathbb{E}(\text{error}) &= \mathbb{E}(\text{FNR}) + \mathbb{E}(\text{FPR}) \\ &= (1 - p_1^{\hat{r}})^{\hat{l}} + 1 - (1 - p_2^{\hat{r}})^{s \cdot \hat{l}}. \end{aligned} \quad (5)$$

Proof: Please refer to Appendix A, available online.

Settings of $\hat{r}$ and $\hat{l}$: According to Theorem 1, the error rate of the discriminator can be controlled by $\hat{r}$ and $\hat{l}$. In particular, increasing $\hat{r}$ would reduce FPR, since two points are considered a ‘collision’ only if all $\hat{r}$ hash values in $\hat{\mathcal{H}}_i$ match; and increasing $\hat{l}$ would reduce FNR, since two points are considered a ‘match’ as long as they are mapped to the same bucket in at least one of the $\hat{l}$ hash groups. To minimize the expected error rate, we set the derivative of $\mathbb{E}(\text{error})$ to zero with respect to $\hat{r}$ by fixing $\hat{r} \cdot \hat{l} = 150$ in our experiments.

C. Update

When the workload shifts over time, the cache must be dynamically updated by evicting stale queries and inserting new queries. To maintain consistency with the cache, the discriminator also needs to be updated accordingly. The pseudo-code of discriminator insertion and deletion is shown in Algorithm 3 and 4, respectively.

- **DISCRIMINATOR-INSERT** inserts a new query q_{new} into D_Q . q_{new} is mapped to $\hat{l}$ bits using the same hash functions $g_1 \circ \hat{\mathcal{H}}_1 - g_{\hat{l}} \circ \hat{\mathcal{H}}_{\hat{l}}$ as in the **DISCRIMINATOR-BUILD** Algorithm. These corresponding bits are then set to 1.
- **DISCRIMINATOR-DELETE** aims to ‘undo’ the procedure of **DISCRIMINATOR-INSERT**. Similarly, q_{evict} is hashed to $\hat{l}$ bits by applying $g_1 \circ \hat{\mathcal{H}}_1 - g_{\hat{l}} \circ \hat{\mathcal{H}}_{\hat{l}}$, and each of these bits is reset to 0.

The time complexity of both Algorithms 3 and 4 is clearly $\mathcal{O}(d\hat{r}\hat{l})$. Since the parameters $\hat{l}$, $\hat{r}$, and d are typically small in practice, they can be treated as constants. As a result, the overall time complexity of both algorithms can be considered as $\mathcal{O}(1)$ in practice.

Analysis: When deleting a query q , one may have a concern that this operation might introduce false negatives (FNs) due to hash collisions with other queries. The following lemma presents the probability of FN caused by deleting a query from cache.

Lemma 1: Given a query q and the cached set $\mathcal{Q} = \{q'_1, \dots, q'_s\}$, let p_i denote the collision probability between q and any query $q'_i \in \mathcal{Q}$ under a single LSH function. The probability of at least one false negative occurring when q is deleted is $1 - \prod_i (1 - p_i^{\hat{r}\hat{l}})$.

Proof: Please refer to Appendix B, available online.

For simplicity, we assume that $p_i = 0.5$ for $1 \leq i \leq s$, $s = 10^6$, $\hat{r} = 10$, and $\hat{l} = 15$, the chance of at least one FN occurring is $1 - (1 - 0.5^{150})^{10^6} \approx 10^{-39}$, which is extremely small. We also empirically evaluated the effect of deletions and found that when deleting 20 k queries, the FNR only increased less than 0.5% across different datasets. In summary, the discriminator is a lightweight data structure enabling fast MMT checking and dynamic workloads with negligible update overhead.

V. THE CACHE

This section details the proposed cache design. We first present the data structure and construction algorithm (Section V-A); next, we describe the query process and pruning strategy (Section V-B); finally, we introduce the eviction rule and the corresponding update mechanism (Section V-C). The workflow of cache is depicted in Fig. 5.

A. Construction

Given a set of historical queries $\mathcal{Q}$ to be cached, the cache $\mathcal{C}_Q$ aims to store $\mathcal{Q}$ along with the kNNs of $\mathcal{Q}$ and support efficient kNN search on the kNN results of $\mathcal{Q}$. A naive approach is to build a classical in-memory ANN index for the union of the kNNs of $\mathcal{Q}$. This approach has several major drawbacks: i) *inefficient query*: although the size of the kNNs of $\mathcal{Q}$ is much smaller than the original dataset $\mathcal{D}$, it is still a large search space

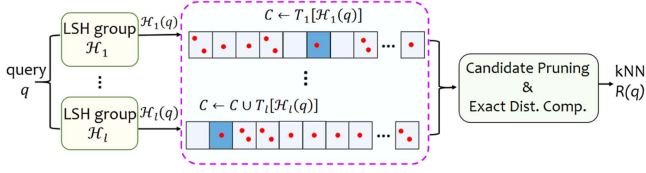


Fig. 5. Cache lookup workflow. The cache employs multiple LSH hash tables $T_1, \dots, T_l$, which map queries to their cached kNN results. For an incoming query q , the system probes each table at location $\mathcal{H}_i(q)$, collects the union of the retrieved buckets as candidates, and then applies projection, pruning, and exact distance computation to determine the final result.

Algorithm 5: Build-Cache $(\{q', \mathcal{R}(q')\}_{q' \in \mathcal{Q}})$.

Input: a query set $\mathcal{Q}$ and their kNN results
Output: the cache $\mathcal{C}_{\mathcal{Q}}$

- 1 Initialize $\mathcal{C}_{\mathcal{Q}} = \{T_1, \dots, T_l\}$ as l empty hash tables
and generate l groups of LSH functions $\mathcal{H}_1, \dots, \mathcal{H}_l$;
- 2 **for** each $q' \in \mathcal{Q}$ **do**
- 3 **for** $i \leftarrow 1$ **to** l **do**
- 4 $T_i[\mathcal{H}_i(q')] \leftarrow \mathcal{R}(q') \cup T_i[\mathcal{H}_i(q')]$;
- 5 **return** $\mathcal{C}_{\mathcal{Q}}$;

and might introduce many distance computations; ii) *inefficient update*: to be effective, $\mathcal{C}_{\mathcal{Q}}$ must dynamically adapt to evolving workloads. However, many traditional indexes are not optimized for frequent insertions and deletions; and (iii) *inconsistency*: it is difficult to maintain consistency between the discriminator and cache, because when stale data points are deleted from the cache, the discriminator should figure out which queries are relevant and should be deleted.

Data Structure: To address these issues, we need to utilize the knowledge of historical queries to facilitate search, i.e., caching both queries and their kNNs to avoid unnecessary distance computations. Specifically, given a query q , we retrieve queries that are similar to q , and their kNNs are regarded as promising candidates of q . However, storing all queries' kNNs would introduce redundancy, as similar queries often share common nearest neighbors. It means a data point might have multiple copies in the cache since it is the common nearest neighbor of multiple similar queries. To address this, we employ LSH-based hash tables as the data structure of the cache, by setting queries as hashed keys, and their kNNs as values stored in buckets. In such cases, similar queries' kNNs are usually hashed to the same bucket, allowing redundant data points within each bucket to be eliminated efficiently.

The pseudo-code of cache construction is presented in Algorithm 5. Specifically, we initialize l hash tables $T_1 - T_l$ (line 1). Each table T_i corresponds to a compound hash function $\mathcal{H}_i = \{h_i^1, \dots, h_i^r\}$ composed of r independent LSH functions based on (1). For each historical query $q' \in \mathcal{Q}$, we first compute its hash value using the function $\mathcal{H}_i$ corresponding to table T_i , mapping q' into an r -dimensional space. The IDs of the k nearest neighbors of q' are then inserted into the bucket indexed by $T_i[\mathcal{H}_i(q')]$ (lines 2–4).

Discussion: Compared to adopting a classical ANN index, we argue that this structure can offer the following advantages: i) *efficient query*: rather than traversing the whole cache to find similar queries of q , the LSH-based hash tables group similar queries into the same bucket. It leads to a nearly constant time of generating candidates; ii) *efficient update*: this index can support efficient updates, as it is easy and fast to delete and insert elements in hash tables; and iii) *consistency*: by tracking the relationship between queries and data points, we can preserve consistency between the discriminator and cache.

Theorem 2: Algorithm 5 achieves a time complexity of $\mathcal{O}(ls(dr + b))$ and a space complexity of $\mathcal{O}(kds)$, where l is the number of hash tables, s is the number of historical queries, r is the number of hash functions per table, and b denotes the bucket size.

Proof: Please refer to Appendix C, available online.

B. Lookup

After the cache is constructed, the remaining problem is how to perform cache lookup. The process consists of two steps: i) *candidate generation*: retrieve historical queries similar to q and collect their corresponding kNN results as candidates. ii) *distance computation*: calculate the distances between q and all candidates, and return the top- k results.

- **Candidate Generation:** Similar to Algorithm 5, the query q is hashed to l buckets using the same hash function groups $\mathcal{H}_1 - \mathcal{H}_l$, that is $T_1[\mathcal{H}_1(q)] - T_l[\mathcal{H}_l(q)]$. All data points stored in these buckets are collected as candidates of q .
- **Distance Computation:** We maintain a priority queue (a max-heap by exact distances) $\mathcal{R}(q)$ that keeps track of the k nearest points to q . For each candidate point o , the distance $\text{dis}(o, q)$ is calculated, and $\mathcal{R}(q)$ is updated accordingly. Specifically, if o is closer to q than the running k -th nearest neighbor, the top element is removed and o is inserted.

Pruning: The time complexity of the second step is $\mathcal{O}(d|C|)$, which can be expensive when the number of candidates $|C|$ or the dimensionality d is large. To further reduce the cost, we introduce an LSH-based pruning strategy that prunes unpromising candidates by projecting both the query and all candidates into a low-dimensional space and calculating distances in this projected space.

We begin by introducing the *projection* function and analyzing the relationship between projected and original distances. Based on this, we formulate an effective *pruning rule* to reduce unnecessary computations.

- **Projection:** First, we generate a compound hash function $g' = \{h'_1, \dots, h'_m\}$ with m random hash functions based on (2). For each candidate point o and the query q , we map them from the original d -dimensional space into the m -dimensional projected space by $g'(\cdot)$. Sun et al. [23] observed that the ratio of the projected distance over the original distance follows a $\chi^2(m)$ distribution. Specifically, for any $x \geq 0$ and point o , we have

$$\Pr[\text{dis}(g'(o), g'(q)) \leq x] = \Psi_m\left(\frac{x^2}{\text{dis}^2(o, q)}\right),$$

Algorithm 6: Cache-Lookup (q, p).

Input: a query q , a probability threshold p
Output: q 's kNN result $\mathcal{R}(q)$

// Step 1: Candidate Generation

```

1  $C \leftarrow \emptyset$ ;
2 for  $i \leftarrow 1$  to  $l$  do
3    $C \leftarrow T_i[\mathcal{H}_i(q)] \cup C$ ;
// Step 2: Distance Computation
4  $\mathcal{R}(q) \leftarrow \emptyset$ ;
5  $g'(C), g'(q) \leftarrow \text{PROJECTION}(C, q)$ ;
6  $g'(C) \leftarrow \text{DIS-COMP-AND-SORT}(g'(C), g'(q))$ ;
7 while  $|g'(C)| > 0$  do
8    $o_i \leftarrow g'(C).\text{POP}()$ ;
9    $dis \leftarrow dis(o_i, q)$ ;
10   $\mathcal{R}(q).\text{INSERT}(o_i, dis^2)$ ;
11  if  $|\mathcal{R}(q)| \geq k$  then
12     $o_j, dis_k^2 \leftarrow \mathcal{R}(q).\text{top}()$ ;
13    if  $\Psi_m\left(\frac{dis(g'(o_i), g'(q))^2}{dis_k^2}\right) \geq p$  then
14      return  $\mathcal{R}(q)$ ;
15 return  $\mathcal{R}(q)$ ;
```

where $\Psi_m(\cdot)$ represents the cumulative distribution function of $\chi^2(m)$.

- **Pruning rule:** Rather than conducting exact distance computations on all the candidates, the above property leads to an effective pruning rule [23]: *if the candidates are sorted in ascending order of their projected distances to q , i.e., if $i_1 \leq i_2$, we have $dis(g'(o_{i_1}), g'(q)) \leq dis(g'(o_{i_2}), g'(q))$. Then the algorithm can terminate when the following condition is satisfied.*

$$\Psi_m\left(\frac{dis^2(g'(o_i), g'(q))}{dis_k^2}\right) \geq p, \quad (6)$$

where $g'(o_i)$ represents the i -th ($i \geq k$) closest point to $g'(q)$ in the projected space; dis_k represents the k -th smallest exact distance between q and all examined points; and p is a specified probability threshold. In the following, we give a theoretical guarantee for the pruning rule.

Lemma 2: Let $\mathcal{R}(q)$ be the kNN result of q returned by the algorithm when the termination condition is satisfied, $\mathcal{R}^*(q)$ be the exact kNN result of q , and o_i (o_i^*) be the i -th ($1 \leq i \leq k$) closest point to q in $\mathcal{R}(q)$ ($\mathcal{R}^*(q)$). For simplicity, we assume that for each $i < k$, the i -th smallest (projected) distance is strictly smaller than the $(i + 1)$ -th smallest (projected) distance. Now we have,

$$\Pr[\mathcal{R}(q) = \mathcal{R}^*(q)] = \Pr[o_k \in \mathcal{R}^*(q)]. \quad (7)$$

Proof: Please refer to Appendix D, available online.

Theorem 3: The pruning rule ensures to return an exact kNN with at least probability of p , i.e., $\Pr[\mathcal{R}(q) = \mathcal{R}^*(q)] \geq p$.

Proof: Please refer to Appendix E, available online.

Algorithm Description: The process of cache lookup is presented in Algorithm 6. In the first step (lines 1–4), q is hashed to

l buckets using $\mathcal{H}_1, \dots, \mathcal{H}_l$, and all points in the corresponding buckets are added into C as candidates. In the second step (lines 6–16), query q and its candidates in C are mapped into an m -dimensional space by calling the PROJECTION procedure (line 7). Then, DIS-COMP-AND-SORT is invoked to compute the projected distances between q and its candidates in C and sort them in ascending order (line 8). Next, we perform a kNN lookup in $g'(C)$ incrementally until all candidates have been visited or the early-termination condition is satisfied (lines 9–16). Specifically, in each iteration, we obtain the nearest unvisited projected point o_i (line 10) and compute the real distance $dis(o_i, q)$. We also maintain the k -th smallest distance dis_k^2 encountered so far (lines 11–12). If the size of $\mathcal{R}(q)$ reaches k , we check if the early-termination condition for o_i is satisfied (lines 13–15). If so, $\mathcal{R}(q)$ is returned as the answer.

Theorem 4 analyzes the worst-case time complexity of the cache lookup operation. In practice, the algorithm exhibits significantly higher efficiency, which stems from two optimizations: early termination of the search and the reuse of precomputed hash values.

Theorem 4: The time complexity of Algorithm 6 is $\mathcal{O}(dmlb)$, where m denotes the projection space dimensionality, l the number of hash tables, and b the bucket size.

Proof: Please refer to Appendix F, available online.

C. Update

This subsection details our cache update mechanism, designed with two objectives: i) adapting to workload changes by flushing stale entries and caching new kNN results, and ii) handling data updates to ensure index-cache consistency. Our discussion primarily focuses on the former, as it addresses the more dynamic aspect of the system.

1) Handling Query Workload Shifts: After the query processing, the cache needs to be updated accordingly. In particular, if query q is missed and cache is full, stale queries and data points need to be evicted from the cache, to make room for q and its kNN. The core challenge in cache updates is twofold: minimizing utility loss during eviction and maintaining logical consistency between the cache and the discriminator.

The intuition behind our strategy is to treat the cache not as a collection of independent points, but as a set of *atomic query-result units*. This perspective addresses the consistency challenge: removing individual points would orphan queries in the discriminator, leading to incorrect results. To strictly enforce this consistency, we prioritize the principle of ATOMIC-EVICT, which mandates evicting an entire query and its complete result set $\mathcal{R}(q)$ as a single unit. Furthermore, we adopt an aggregate utility metric, where the importance of a cached query is derived directly from the combined utility of its k nearest neighbors.

Data Structure: To implement these principles, we maintain two scoring structures: L_D for data points and L_Q for queries. For a data point o , $L_D(o)$ records its utility (e.g., access frequency or recency). For a query q , we define its aggregate importance $L_Q(q)$ as: $L_Q(q) = \sum_{o \in \mathcal{R}(q)} L_D(o)$. To facilitate efficient retrieval of the minimum score, L_Q is maintained using a red-black tree, allowing for logarithmic time updates.

Algorithm 7: Cache-Update ($q_{\text{new}}, \mathcal{R}(q_{\text{new}})$).

Input: Query q_{new} with its kNN results $\mathcal{R}(q_{\text{new}})$
Output: Updated cache state with q_{new} inserted (and stale entries evicted if necessary)

```

1 UPDATE-REFERENCE-SCORES( $q_{\text{new}}, \mathcal{R}(q_{\text{new}})$ );
2 if CacheIsFull and Discriminator-Lookup( $q_{\text{new}}$ ) =
  false then
3    $q_{\text{evict}} \leftarrow \arg \min_{q \in L_Q} L_Q(q)$ ;
4   ATOMIC-EVICT( $q_{\text{evict}}, \mathcal{R}(q_{\text{evict}})$ );
5 if Discriminator-Lookup( $q_{\text{new}}$ ) = false then
6   ATOMIC-INSERT( $q_{\text{new}}, \mathcal{R}(q_{\text{new}})$ );

```

Eviction Rule: When cache overflow occurs, we must select a victim to make room for new content. Based on the aggregate utility metric, our rule selects the query with the lowest score:

$$q_{\text{evict}} \leftarrow \arg \min_{q \in L_Q} L_Q(q). \quad (8)$$

This strategy balances two key objectives: on one hand, it minimizes the global utility loss incurred by evicting the kNN points associated with q_{evict} ; on the other, it ensures that the query and its results are removed atomically, thereby preserving the logical consistency of the discriminator.

Cache Update: Algorithm 7 details the update process for a new query q_{new} . First, UPDATE-REFERENCE-SCORES refreshes utility statistics (L_D and L_Q) using q_{new} and its kNN results (line 1). If the cache is full and q_{new} is a miss, a victim q_{evict} is selected via the eviction policy (lines 2–3), and ATOMIC-EVICT then removes q_{evict} and its associated kNN points from the cache (line 4). Subsequently, if q_{new} is a miss, ATOMIC-INSERT adds q_{new} and its kNN points to the cache (line 6); hits require no further action. The atomic operations encapsulate low-level manipulations of the discriminator and LSH buckets, whose details are omitted for brevity.

Note that our cache supports two update modes: per-query and batch. The per-query mode offers higher adaptability to workload shifts but incurs greater update overhead, making it suitable for dynamic environments with frequent query pattern changes. In contrast, the batch mode reduces update frequency at the cost of reduced adaptability, which is more practical for stable workloads where query patterns evolve slowly.

2) *Handling Data Updates:* The core of our cache consistency mechanism stems from a simple observation: regardless of operation type, the graph index is modified only through updates to the adjacency lists of the points involved. Consequently, a cached point is considered impacted *if and only if its neighborhood has changed*; otherwise, it remains unaffected. Based on this insight, we design a *unified update handling mechanism* that treats insertions and deletions identically, eliminating the need for operation-specific logic.

Our core idea is to periodically probe only the cached hot points to detect changes in their adjacency lists. When a hot point o 's neighborhood has been altered, it is flagged as *impacted*. Subsequently, any cached query whose kNN result contains more than a threshold number (e.g., 3 for $k = 10$) of impacted points is

TABLE I
DATASET STATISTICS

Dataset	Size	Dimensions	Data Type
Last.fm	292,385	64	Audio
SIFT	1,000,000	128	Image
GloVe	1,183,514	50	Text
DEEP	10,000,000	96	Image
Text2Image	10,000,000	200	Image
SPACEV	10,000,000	100	Text
BIGANN	10,000,000	128	Image

also identified as *impacted*. All such impacted points and queries are then evicted from the cache, ensuring consistency.

To track these changes efficiently, each point in the graph index is assigned a *modification bit* (initialized to 0). During each scan, if a hot point's modification bit is 1, we mark it as impacted and reset the bit to 0. A bit value of 0 indicates an unmodified neighborhood, requiring no action.

This design offers two key advantages. First, it provides a *unified treatment* of insertions and deletions, simplifying implementation and reducing complexity. Second, and more importantly, it *decouples* cache maintenance from index updates: rather than checking the cache upon every data modification, an operation whose cost would scale with both update frequency and cache size, we periodically inspect only the cached content itself. This approach bounds the consistency overhead to the cache size rather than the update rate, making it particularly suitable for high-throughput dynamic environments.

VI. PERFORMANCE EVALUATION

In this section, we evaluate the performance of ANN-CACHE compared with existing disk-resident ANN methods over six real-world benchmark datasets.

A. Experimental Setting

Datasets: We use seven real-world benchmark datasets of various sizes, data types, and dimensions, as shown in Table I.

Workloads: For Last.fm, SIFT, GloVe, and BIGANN, we adopt its provided real-world query workloads. Specifically, Last.fm comprises 50 k unique queries, while SIFT, GloVe and BIGANN each provide 10 k queries. Following the methodology in [16], we replicate these queries to achieve the target evaluation scale of up to 100 k. For the other datasets (DEEP, Text2Image, and SPACEV), we generate synthetic workloads by sampling 20 k data points from a Zipfian distribution ($\alpha = 1.5$) and perturbing each sample for variability, as in [24]. The resulting query set is then partitioned into historical and test subsets in a 3:2 ratio.

Algorithms: We integrate ANN-CACHE into three state-of-the-art disk-based ANN algorithms: DISKANN, STARLING, and SPANN, naming the variants DISKANN+, STARLING+, and SPANN+, respectively. Note that, DISKANN maintains a cache storing vertices near the entry point to accelerate search, whereas DISKANN+ replaces this specific cache with our proposed ANN-CACHE. For simplicity, we refer to the original baselines as ANN, and the variants enhanced with ANN-CACHE as ANN+.

TABLE II
QPS, I/O, AND RECALL COMPARISONS FOR ALL ALGORITHMS ACROSS SIX DATASETS

		DISKANN	DISKANN+	INCREASE	STARLING	STARLING+	INCREASE	SPANN	SPANN+	INCREASE
Last.fm	QPS	313	1367	4.37×	442	1380	3.12×	1522	4568	3.00×
	I/O	62	9	6.89×	40	6	6.67×	14	2	7.00×
	recall	0.9	0.9127	0.013	0.940	0.944	0.004	0.9345	0.9356	0.001
SIFT	QPS	736	2145	2.91×	834	3017	3.62×	895	4789	5.35×
	I/O	24	5	4.80×	32	7	4.57×	29	4	7.25×
	recall	0.9508	0.9558	0.005	0.939	0.941	0.002	0.9346	0.9376	0.003
GloVe	QPS	395	2001	5.07×	676	2968	4.39×	840	4907	5.84×
	I/O	46	9	5.11×	24	4	6.00×	29	4	7.25×
	recall	0.893	0.904	0.011	0.947	0.935	-0.012	0.924	0.927	0.003
DEEP	QPS	432	1583	3.66×	393	1808	4.6×	642	2585	4.03×
	I/O	42	10	4.20×	41	7	5.85×	39	6	6.50×
	recall	0.943	0.955	0.012	0.976	0.971	-0.005	0.901	0.907	0.006
Text2Image	QPS	315	1452	4.61×	470	2584	5.50×	95	886	9.33×
	I/O	57	11	5.18×	37	7	5.29×	98	15	6.53×
	recall	0.938	0.944	0.006	0.949	0.951	0.002	0.913	0.922	0.009
SPACEV	QPS	461	1883	4.08×	816	4830	5.92×	317	1650	5.21×
	I/O	42	9	4.67×	17	3	5.67×	63	10	6.30×
	recall	0.928	0.935	0.007	0.906	0.926	0.020	0.943	0.955	0.012

Without specific demonstration, we adopt STARLING+ as the default ANN+ method.

- DISKANN [9] introduces a Vamana graph index with smaller diameter to minimize the number of sequential disk reads.
- STARLING [11] enhances DISKANN by optimizing the data layout on SSDs and improving the search algorithm to achieve better performance.
- SPANN [10] builds high-quality posting lists using a hierarchical balanced clustering algorithm and a closure-based assignment strategy. It also adopts a query-aware mechanism to achieve both high recall and low latency.

Metrics: We adopt four metrics to evaluate the performance of the algorithms, i) QPS, i.e., the number of handled queries per second, to measure the efficiency; ii) I/O, i.e., the number of disk I/Os for each query, to measure the efficiency of disk-resident algorithms; iii) recall to measure the accuracy; and iv) memory overhead to captures the total space consumption, primarily comprising cached data vectors and associated cache structures.

Parameter Settings: By default, we set $k = 10$. For ANN-CACHE, we set the length of bit array as $\hat{m} = 1,000,000$, the number of LSH functions in $\hat{\mathcal{H}}_i$ as $\hat{r} = 10$, the number of $\hat{\mathcal{H}}_i$ s as $\hat{l} = 15$, and the probability threshold $p = 0.99$. We adopt LFU policy for static workloads and LRU policy for dynamic workloads. The parameters of baselines are set to default values as suggested by the original authors or otherwise tuned to achieve the best performance on each dataset if not specified. For DISKANN and STARLING, we set the graph construction parameters to be $R = 32$, $L = 128$, $\alpha_1 = 1.0$ and $\alpha_2 = 1.2$. For SPANN, we choose $\text{MaxCheckForRefineGraph} = 20,000$ to control how many nodes each node will visit during graph refinement and $\text{MaxCheck} = 16,324$ to control the quality of approximate nearest neighbors. For other parameters, we adopt the default configuration.

Setup: We conduct all experiments on a PC with AMD Ryzen7 4800H 4-core 2.9 GHz CPU (hyper-threading to 8 threads) and 32 GB RAM. All C++ source codes are compiled by g++ 11.4.0 with -O3 optimization under Ubuntu 20.04 LTS.

B. Overall Effectiveness

Exp-1. Search Performance: To evaluate the effectiveness of ANN-CACHE in facilitating ANN search, we compare the ANN-CACHE-enhanced methods (DISKANN+, STARLING+, and SPANN+) with their respective baselines (DISKANN, STARLING, and SPANN) across six real-world datasets. Table II depicts the QPS, recall and I/O. We summarize our key findings as follows.

i) All ANN+ methods significantly outperform their original counterparts in terms of QPS across all datasets. In particular, DISKANN+ achieves $2.9\times - 5.1\times$ higher QPS than DISKANN, STARLING+ achieves $3.1\times - 5.9\times$ higher than STARLING, and SPANN+ achieves $3.0\times - 9.3\times$ higher than SPANN. These results confirm our observation in Section I, i.e., caching historical queries and their results can significantly improve query efficiency.

ii) A similar trend is observed in I/O cost. In most cases, the QPS improvement is slightly smaller than the corresponding reduction in I/O cost. This is because a cache miss (i.e., when the discriminator returns false or the cache provides unsatisfactory results) incurs an extra cost from accessing the discriminator and cache. However, in some cases, the QPS improvement exceeds the reduction in I/O cost. For example, on Text2Image, the QPS improvement of SPANN+ exceeds its I/O improvement by approximately $2.8\times$. This suggests that our in-memory search process is more efficient than the in-memory search of the baselines in certain scenarios.

iii) ANN+ methods achieves comparable or even higher recall than ANN methods. The results verify the observation in Section I that exploring neighbors across all similar queries yields accurate kNN results in most cases, and in some instances, even uncovers more neighbors than traditional ANN search.

To sum up, ANN-CACHE is highly effective in enhancing disk-resident ANN methods by significantly improving query efficiency, while also achieving gains in accuracy.

Exp-2. Memory Overhead: Table III displays the memory overhead of all methods across six datasets under the same query workload. The result shows that the memory overhead incurred by ANN-CACHE remains consistently small (< 50 MB). Given

TABLE III
MEMORY OVERHEAD COMPARISONS OF ALL ALGORITHMS ACROSS SIX DATASETS (MB)

	DISKANN	DISKANN+	INCREASE	Starling	STARLING+	INCREASE	SPANN	SPANN+	INCREASE
Last.fm	269	215	-54	202	245	43	67	28	39
SIFT	387	312	-75	353	381	28	104	75	29
GloVe	282	201	-81	226	263	37	82	42	40
DEEP	1101	1011	-90	1168	1193	25	147	121	26
Text2Image	1302	1192	-110	1389	1437	48	293	245	48
SPACEV	429	343	-86	490	500	10	115	105	10

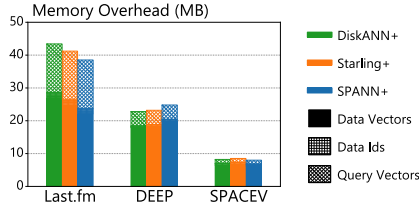


Fig. 6. Memory breakdown of ANN-Cache.

the substantial improvements in query performance, this additional memory cost is totally worth paying. We also find that the memory overhead of ANN-CACHE mainly depends on dimensionality and data format, *e.g.*, Text2Image ($d = 200$) results in higher memory than other lower-dimensional datasets; and the float32-type dataset DEEP ($d = 96$) requires approximately $2.5\times$ memory than the int8-type dataset SPACEV ($d = 100$).

Interestingly, DISKANN+ exhibits lower memory overhead than the original DISKANN across all datasets. By replacing DISKANN's native cache with our more efficient ANN-CACHE, we achieve net memory savings alongside gains in accuracy and search speed, confirming ANN-CACHE as a lightweight, high-performance drop-in replacement.

Exp-3. Memory Breakdown: Fig. 6 provides a detailed breakdown of memory consumption for ANN-CACHE on Last.fm, DEEP, and SPACEV across three index variants (DISKANN+, STARLING+, and SPANN+). Three components are visually distinguished: data vectors (solid bars), query vectors (hatched bars), and data IDs (grid-patterned bars). Note that the discriminator incurs a small fixed memory cost (<2 MB) and is omitted from the figure. Two key observations emerge. First, a consistent predominant pattern is observed: data vectors dominate memory usage across all configurations, accounting for 55%–74% of the total footprint. Query vectors contribute 12%–37%, while data IDs constitute a minor fraction ($<7\%$). Second, and more importantly, the memory breakdown exhibits remarkable stability across different index structures on the same dataset, suggesting that ANN-CACHE's memory footprint is dataset-driven and index-agnostic.

C. Ablation Study

We conduct ablation experiments to validate the effectiveness of our design: the discriminator, the pruning strategy, and the LSH-based cache structure.

Exp-4. Discriminator Validation: The discriminator serves as the gatekeeper that determines whether a query can be served from the cache. To validate its effectiveness, we analyze its

TABLE IV
PERFORMANCE OF THE DISCRIMINATOR

	Last.fm	SIFT	GloVe	DEEP	Text2Image	SPACEV
FPR	0.002	0.001	0.003	0	0.001	0.001
FNR	0.055	0.008	0.011	0.010	0.013	0.020
error rate	0.057	0.009	0.014	0.010	0.014	0.021
latency (ms)	0.014	0.028	0.012	0.023	0.037	0.025

TABLE V
REDUCTION IN DISTANCE COMPARISON OPERATIONS

	Last.fm	SIFT	GloVe	DEEP	Text2Image	SPACEV
DISKANN+	15.2%	29.8%	13.3%	22.7%	37.4%	23.4%
STARLING+	16.8%	31.2%	11.8%	23.5%	36.9%	26.4%
SPANN+	13.6%	28.4%	12.1%	24.8%	35.5%	28.6%

error rates and latency across all datasets. Table IV reports the discriminator's performance in terms of false positive rate (FPR), false negative rate (FNR), error rate (FNR+FPR) and latency. The discriminator achieves extremely low error rates (ranging from 0.01 to 0.06) with negligible time overhead (under 0.037 ms) across all datasets. We also find that FNR is the primary factor affecting the error rate. Upon examining false-negative-inducing queries, we observe a consistent pattern: the vast majority involve cases where merely 1 or 2 neighbors are cached. This finding suggests our discriminator effectively filters suboptimal results by returning false for queries whose nearest neighbors are predominantly absent from the cache.

Exp-5. Pruning Validation: To accelerate query execution, we introduce an LSH-based pruning strategy that eliminates unnecessary exact distance computations for non-promising candidates. We validate this component by measuring the reduction in distance comparison operations. Table V reports the average percentage reduction in distance comparison operations achieved by ANN+ methods across all datasets. The pruning strategy reduces distance computations by 11.8%–37.4% across datasets, yielding an average speedup of $1.21\times$ when SIMD optimizations are disabled. These results confirm the effectiveness of our pruning strategy in lowering the distance computation cost. In addition, the effectiveness of pruning varies with data distribution. For instance, the strategy proves more effective on SIFT, DEEP, Text2Image and SPACEV than on Last.fm and GloVe.

Exp-6. Cache Structure Validation: Having validated the individual components, we now turn to the cache structure itself. To establish its necessity, we compare ANN-CACHE against classic caching methods. We augment the STARLING index with three caching strategies: i) our proposed ANN-CACHE, ii) Least Frequently Used (LFU), and iii) Least Recently Used (LRU), denoted as STARLING+ANN-CACHE, STARLING+LFU, and STARLING+LRU, respectively. To ensure fair comparison,

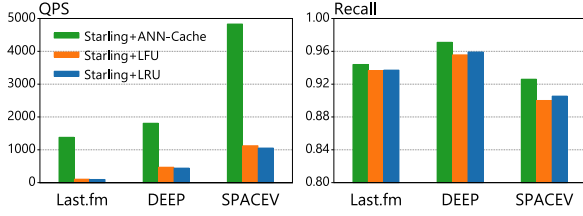


Fig. 7. Comparison with LFU and LRU.

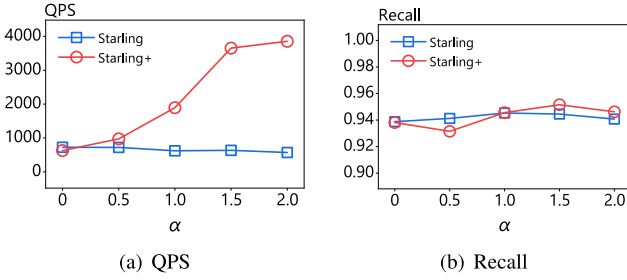


Fig. 8. Impact of query skewness.

all variants adopt the same approximate-match ($\approx$) condition for cache hits and identical parameter settings. As shown in Fig. 7, ANN-CACHE consistently outperforms both LFU and LRU across all datasets, achieving substantial gains in efficiency while delivering modest improvements in accuracy. Specifically, it achieves QPS improvements of $12.3\times$ on Last.fm, $3.9\times$ on DEEP, and $4.3\times$ on SPACEV. This performance gap stems from a fundamental difference in lookup mechanisms. LFU and LRU require a linear scan over the entire cache to identify matches. In contrast, ANN-CACHE leverages its LSH-based design to enable near-constant-time lookups independent of cache size while also exploring a broader set of relevant neighbors.

D. Sensitivity Analysis

We now examine how ANN-CACHE's performance varies with workload characteristics and system parameters, including query skewness, cache configuration, and discriminator parameters.

Exp-7. Query Skewness: We first investigate how the query distribution affects caching effectiveness. By varying the Zipf parameter α from 0 to 2 on DEEP, we compare STARLING+ against the baseline STARLING. Fig. 8 presents the results. As shown in Fig. 8(a), the baseline STARLING maintains stable QPS across all α values, as its performance is independent of query distribution. In contrast, STARLING+ exhibits significant QPS improvement as skewness increases, reaching $4.3\times$ speedup at $\alpha = 1.5$. This gain stems from higher cache hit rates under skewed workloads, which reduce I/O cost. Fig. 8(b) shows that recall remains largely stable, with a slight fluctuation at $\alpha = 0.5$. This behavior reflects the interplay between cache utilization and accuracy. At $\alpha = 0$, the random workload yields minimal cache hits, so accuracy is determined by the disk-resident index. As α increases to 0.5, more queries are served from the cache, but insufficient similar queries may temporarily degrade recall.

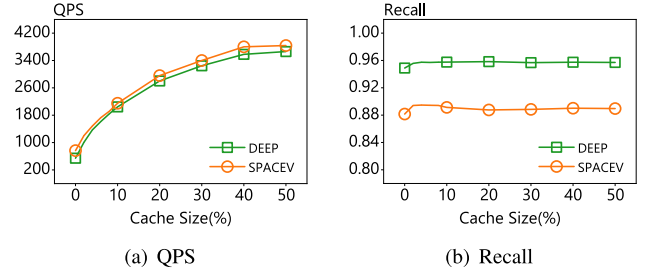
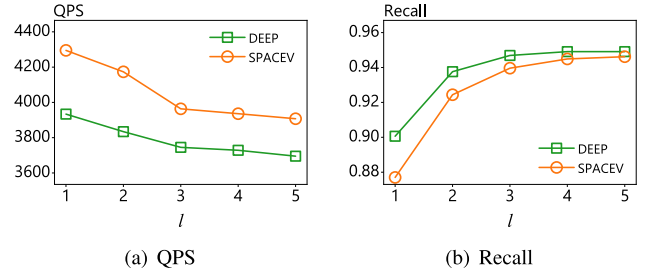


Fig. 9. Impact of cache size.

Fig. 10. Impact of hash functions l .

When $\alpha > 1$, sufficient similar queries accumulate, enabling STARLING+ to match or exceed baseline accuracy. These findings confirm that ANN-CACHE delivers substantial gains on skewed workloads while maintaining accuracy across the spectrum.

Exp-8. Cache size: Fig. 9 shows the effect of varying the cache-to-dataset ratio from 0% to 50% on DEEP and SPACEV. QPS exhibits a characteristic growth pattern: rapid initial improvement (a 10% cache delivers $2.8\times$ – $3.7\times$ speedup, capturing about 50% of the total gain), followed by diminishing returns and saturation at a 40% ratio ($5.0\times$ – $6.6\times$ speedup). Recall remains stable throughout, confirming accuracy robustness. This saturation behavior yields a practical tuning strategy: the optimal cache size corresponds to the point where QPS growth plateaus, indicating that the working set is fully captured. Since this point varies with workload characteristics, we recommend a dynamic approach that monitors QPS while incrementally expanding cache capacity—a significant QPS drop signals undersizing, while plateauing indicates optimal size.

Exp-9. Hash Functions: Fig. 10 illustrates the trade-off introduced by l . As l increases from 1 to 5, QPS decreases due to more candidate points and longer distance computation, while recall improves as the expanded search space yields better results. Accuracy gains diminish after $l \geq 4$, where recall stabilizes. This finding suggests a lightweight grid search for l : start from a minimal value and increment until accuracy stabilizes, balancing efficiency and precision dynamically.

Exp-10. Discriminator Parameters: We analyze the sensitivity of the discriminator to its two parameters: the number of hash functions per table $\hat{r}$ and the number of hash tables $\hat{l}$. Fig. 11 reports the error rate (FPR + FNR) under varying configurations. With $\hat{l} = 20$ fixed, increasing $\hat{r}$ from 6 to 16 reduces collision probability, causing FPR to decrease and FNR to increase monotonically. The overall error rate exhibits a U-shaped pattern,

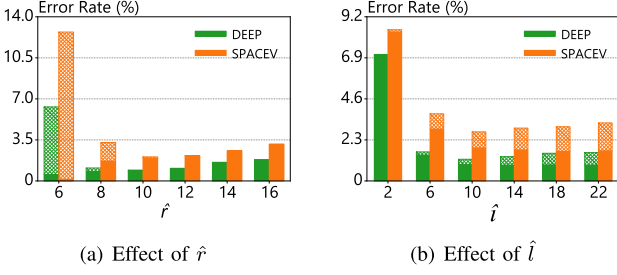
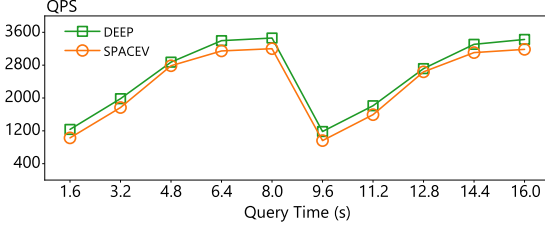
Fig. 11. Impact of discriminator parameters $\hat{r}$ and $\hat{l}$.

Fig. 12. Performance under shifting workloads.

reaching its minimum at $\hat{r} = 10$ on both datasets. With $\hat{r} = 8$ fixed, increasing $\hat{l}$ from 2 to 22 has the opposite effect: more hash tables increase collision probability, raising FPR while reducing FNR. The error rate again minimizes at $\hat{l} = 10$. To circumvent exhaustive manual search in discriminator tuning, we exploit an empirical observation: fixing the product $\hat{r} \times \hat{l}$ in proximity to a constant (e.g., 150) yields consistently robust performance. By imposing the constraint $\hat{r} \cdot \hat{l} = 150$, the optimal parameter $\hat{r}$ can be analytically derived by setting the derivative of the expected error rate $\mathbb{E}(\text{error})$ to zero.

E. Robustness Analysis

We evaluate ANN-CACHE under three challenging scenarios: workload shifts, continuous updates, and non-repetitive queries.

Exp-11. Workload Shifts: We evaluate ANN-CACHE's adaptability to workload changes on DEEP and SPACEV. Two distinct workloads (25 k queries each) are executed sequentially for 9.6 seconds each, starting with an empty cache. The shift makes the cached data stale, requiring eviction and insertion. Fig. 12 illustrates the QPS evolution of STARLING+ over time. In the first phase, QPS increases and stabilizes as hot data populate the cache. After the workload shift, QPS drops sharply as previously cached data becomes stale. As new queries are processed, QPS recovers steadily in proportion to the number of accessed queries, ultimately reaching the same performance level. These results demonstrate that ANN-CACHE adapts to dynamic workloads with minimal disruption.

Exp-12. Continuous Updates: We evaluate ANN-CACHE under continuous insertions and deletions by integrating it into FRESHDISKANN, a dynamic disk-based index, yielding FRESHDISKANN+. The index is initialized with 90% of the dataset, followed by 10 batches. In each batch, we randomly delete 1% of vectors, insert 1% new vectors, and execute 1000 queries. As shown in Fig. 13(a), FRESHDISKANN+ closely tracks the baseline's recall trajectory, with a marginal gap of

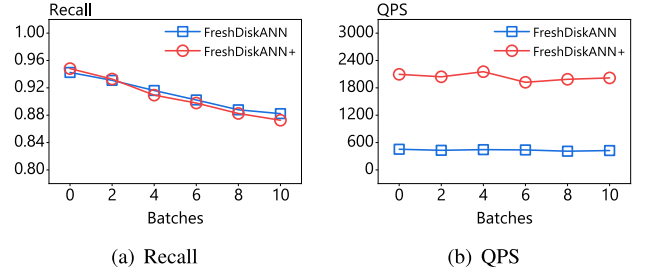


Fig. 13. Performance under dynamic workloads.

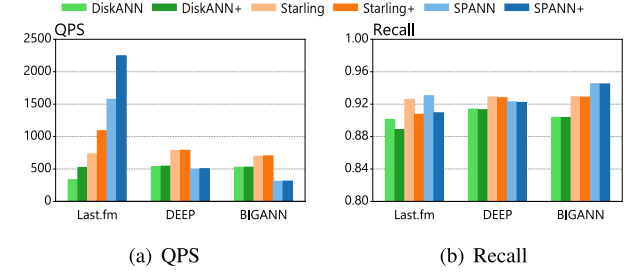


Fig. 14. Performance under non-repetitive query workloads.

less than 0.01, indicating that ANN-CACHE preserves index consistency as data evolves. Meanwhile, Fig. 13(b) shows that it maintains stable high throughput, delivering a $4.2\times$ – $5.2\times$ speedup over FRESHDISKANN. Together, these results confirm that ANN-CACHE delivers substantial throughput gains without compromising recall accuracy under dynamic updates.

Exp-13. Non-repetitive Workloads: We evaluate the robustness of ANN-CACHE on three datasets, DEEP, BIGANN, and Last.fm, using their official non-repetitive query workloads. This eliminates any advantage from caching identical or similar queries. The QPS and recall are reported in Fig. 14. Two key observations emerge. First, on Last.fm, where a small subset of queries incidentally share a portion of their k-nearest neighbors, ANN-CACHE boosts QPS by 42%–56% with only 2% recall loss. This demonstrates that even with limited cache utility, our approach exploits spatial locality inherent in the data. Second, on DEEP and BIGANN, where queries share virtually no neighbors and cache hits are nearly absent, the impact of ANN-CACHE is minimal: QPS varies by $< 0.8\%$ and recall drops by $< 0.04\%$ across all indexes. These results confirm two essential properties of ANN-CACHE: it is opportunistic, capturing significant gains under favorable workloads, and robust, imposing negligible overhead even with non-repetitive queries. This graceful degradation ensures safe deployment across diverse applications.

VII. RELATED WORK

ANN Search: ANN search is a well-established problem that has been extensively studied in the database community [25]. Most existing methods are designed to reside entirely in memory (Please see [26] for survey). However, as data volume increases, these memory-resident algorithms may suffer from increased latency, as it becomes impractical to load the entire dataset into memory. Recently, several disk-based ANN methods [9], [10], [11] have been proposed to enable efficient search on large-scale

datasets. Since query performance is primarily dominated by I/O efficiency, these methods focus on minimizing disk I/O operations. For instance, DISKANN [9] reduces the number of sequential disk reads by constructing a Vamana graph index with smaller diameter, while STARLING [11] further lowers I/O cost by optimizing data layout on disk and introducing a memory-resident navigation graph. Inverted-index-based methods such as IVFADC [27] and FAISS [28] partition the vector space into K Voronoi regions using kmeans clustering and perform search only within regions close to the query. SPANN [10], a more recent method, balances the lengths of posting lists via hierarchical balanced clustering and duplicates boundary vectors across neighboring clusters to improve search coverage.

Recent research has also focused on supporting dynamic updates in large-scale vector indexes. FRESHDISKANN [29] introduces the first graph-based index that supports real-time insertions and deletions by maintaining an in-memory buffer for recent updates and periodically merging it with an SSD-resident index using a write-efficient two-pass StreamingMerge algorithm. Similarly, SPFRESH [30] supports in-place vector updates through LIRE, a lightweight incremental rebalancing protocol that only reassigns boundary vectors in the nearby partitions, thereby adapting to data distribution shifts without costly global rebuilds. Both approaches achieve low-latency query processing while maintaining high recall under continuous update streams.

Remark: However, most existing methods either ignore the potential of caching for performance enhancement, or employ specialized caching strategies that are tightly coupled with their methods and thus difficult to generalize. In contrast, our proposed ANN-CACHE is orthogonal to these indexing techniques and can be broadly applied as a plug-in to accelerate various disk-based ANN methods.

Caching: Our work is also related to caching. Cache-conscious query processing mainly adopts classical data structure to organize cached data, including Bloom Filters, hash tables, trees, hybrid structures, and so on. The Dynamic Blocked Bloom Filter [31] allows each block to grow dynamically as the number of elements inserted increases, which enables it to maintain a low error rate without needing to scale the size of every block by a fixed percentage. Cavast [32] redesign the hash indexing structure to effectively improve the utilization of the limited cache space. CHIME [33] combines B+ trees with hopscotch hashing to achieve low cache consumption and low read amplifications simultaneously.

Remark: These existing structures are primarily designed for traditional database workloads without exploiting ANN-specific characteristics. In contrast, our design leverages properties of ANN search to achieve fundamental optimizations.

VIII. CONCLUSION AND FUTURE WORK

We propose a caching-enhanced approach ANN-CACHE that aims at reducing I/O cost of disk-resident ANN methods. The key novelty of ANN-CACHE is to utilize the knowledge of past queries to avoid unnecessary index lookup. To this end, we propose a light-weight discriminator to perform cache hit checking and an LSH-based cache that stores past queries along

with their results to support retrieving similar queries and their corresponding results. Experiments show that ANN-CACHE can significantly improve the efficiency of existing disk-resident ANN methods.

We identify several directions for future research. i) We plan to extend ANN-CACHE to distributed environments using a shared-nothing design, where each node independently maintains its local index and local cache. A query router identifies relevant nodes and broadcasts queries; each node first probes its local cache before accessing the disk-resident index, and returns results to the router for aggregation. This architecture preserves the transparency of ANN-CACHE while enabling horizontal scaling. ii) To improve real-world deployability, we will enhance ANN-CACHE to support diverse similarity metrics (e.g., cosine, Jaccard) via pluggable LSH families (e.g., SimHash, MinHash), with automatic hyperparameter tuning to eliminate manual configuration. Furthermore, we will develop adaptive mechanisms to dynamically tune cache admission and eviction policies based on online query pattern analysis, ensuring robust performance under varying workloads and access skew.

REFERENCES

- [1] A. Kosuge and T. Oshima, "An object-pose estimation acceleration technique for picking robot applications by using graph-reusing k-NN search," in *Proc. Int. Conf. Graph Comput.*, 2019, pp. 68–74.
- [2] Z. Yang, B. Zheng, X. Wang, G. Li, and X. Zhou, "minIL: A simple and small index for string similarity search with edit distance," in *Proc. Int. Conf. Data Eng.*, 2022, pp. 565–577.
- [3] C. Li, M. Zhang, D. G. Andersen, and Y. He, "Improving approximate nearest neighbor search through learned adaptive early termination," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 2539–2554.
- [4] J. Li et al., "A general and efficient querying method for learning to hash," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2018, pp. 1333–1347.
- [5] Y. Tian, Z. Yue, R. Zhang, X. Zhao, B. Zheng, and X. Zhou, "Approximate nearest neighbor search in high dimensional vector databases: Current research and future directions," *IEEE Data Eng. Bull.*, vol. 46, no. 3, pp. 39–54, Mar. 2023.
- [6] K. Hajebi, Y. Abbasi-Yadkori, H. Shahbazi, and H. Zhang, "Fast approximate nearest-neighbor search with k-nearest neighbor graph," in *Proc. Int. Joint Conf. Artif. Intell.*, 2011, pp. 1312–1317.
- [7] M. Muja and D. G. Lowe, "Scalable nearest neighbor algorithms for high dimensional data," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 36, no. 11, pp. 2227–2240, Nov. 2014.
- [8] J. Wang, T. Zhang, J. Song, N. Sebe, and H. T. Shen, "A survey on learning to hash," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 769–790, Apr. 2018.
- [9] S. J. Subramanya, Devvrit, R. Kadekodi, R. Krishnaswamy, and H. V. Simhadri, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 1–11.
- [10] Q. Chen et al., "SPANN: Highly-efficient billion-scale approximate nearest neighbor search," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 5199–5212.
- [11] M. Wang et al., "Starling: An I/O-efficient disk-resident graph index framework for high-dimensional vector similarity search on data segment," in *Proc. ACM Manage. Data*, vol. 2, no. 1, pp. 1–27, 2024.
- [12] K. Wang and F. Chen, "Catalyst: Optimizing cache management for large in-memory key-value systems," *Proc. VLDB Endow.*, vol. 16, no. 13, pp. 4339–4352, 2023.
- [13] X. Wang, P. Jin, Y. Luo, and Z. Chu, "Range cache: An efficient cache component for accelerating range queries on LSM-based key-value stores," in *Proc. Int. Conf. Data Eng.*, 2024, pp. 488–500.
- [14] J. Ding, V. Nathan, M. Alizadeh, and T. Kraska, "Tsunami: A learned multi-dimensional index for correlated data and skewed workloads," *Proc. VLDB Endow.*, vol. 14, no. 2, pp. 74–86, 2020.

- [15] T. Bertin-Mahieux, D. P. W. Ellis, B. Whitman, and P. Lamere, "The million song dataset," in *Proc. Int. Soc. Music Inf. Retrieval Conf.*, 2011, pp. 591–596.
- [16] V. Mageirakos, B. Wu, and G. Alonso, "Cracking vector search indexes," 2025, *arXiv:2503.01823*.
- [17] S. Agarwal et al., "Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation," 2025, *arXiv:2502.15734*.
- [18] C. Fu and D. Cai, "EFANNA: An extremely fast approximate nearest neighbor search algorithm based on kNN graph," 2016, *arXiv:1609.07228*.
- [19] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [20] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. Conf. Symp. Theory Comput.*, 1998, pp. 604–613.
- [21] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, "Locality-sensitive hashing scheme based on P-stable distributions," in *Proc. 20th Annu. Symp. Comput. Geometry*, 2004, pp. 253–262.
- [22] Q. Huang, J. Feng, Y. Zhang, Q. Fang, and W. Ng, "QALSH: Query-aware locality-sensitive hashing for approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 9, no. 1, pp. 1–12, 2015.
- [23] Y. Sun, W. Wang, J. Qin, Y. Zhang, and X. Lin, "SRS: Solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index," *Proc. VLDB Endow.*, vol. 8, no. 1, pp. 1–12, 2014.
- [24] K. Ren, Y. Kwon, M. Balazinska, and B. Howe, "Hadoop's adolescence: An analysis of Hadoop usage in scientific workloads," *Proc. VLDB Endow.*, vol. 6, no. 10, pp. 853–864, 2013.
- [25] W. G. Aref et al., "A video database management system for advancing video database research," in *Proc. Int. Workshop Multimedia Inf. Syst.*, 2002, pp. 1–15.
- [26] H. V. Simhadri et al., "Results of the NeurIPS '21 challenge on billion-scale approximate nearest neighbor search," in *Proc. NeurIPS Competitions Demonstrations Track*, 2022, pp. 177–189.
- [27] H. Jégou, R. Tavenard, M. Douze, and L. Amsaleg, "Searching in one billion vectors: Re-rank with source coding," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2011, pp. 861–864.
- [28] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.
- [29] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri, "FreshDiskANN: A fast and accurate graph-based ANN index for streaming similarity search," 2021, *arXiv:2105.09613*.
- [30] Y. Xu et al., "SPFresh: Incremental in-place update for billion-scale vector search," in *Proc. 29th Symp. Operating Syst. Princi.*, 2023, pp. 545–561.
- [31] T. Kaler, "Cache efficient bloom filters for shared memory machines," 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:17933140>
- [32] K. Wang, J. Liu, and F. Chen, "Put an elephant into a fridge: Optimizing cache efficiency for in-memory key-value stores," in *Proc. VLDB Endow.*, vol. 13, no. 9, pp. 1540–1554, 2020.
- [33] X. Luo, J. Shen, P. Zuo, X. Wang, M. R. Lyu, and Y. Zhou, "CHIME: A cache-efficient and high-performance hybrid index on disaggregated memory," in *Proc. Symp. Operating Syst. Princi.*, 2024, pp. 110–126.



Lingli Li received the PhD degree from the Harbin Institute of Technology, China. She is currently a professor with the School of Computer Science and Technology, Heilongjiang University, China. She has authored or co-authored more than 30 papers in refereed journals and conferences, including *IEEE Transactions on Knowledge and Data Engineering*, *Information Sciences*, and *CIKM*. Her research interests include Big Data management and data quality. Her work has been cited more than 230 times.



Zhanyu He is currently a postgraduate degree with the Computer Technology, Heilongjiang University, China. His research interests include similarity search in high-dimensional spaces and AI4DB.



Zhuo Zhang is currently working toward the undergraduate degree with the Internet of things Engineering, Heilongjiang University, China. His research interests include similarity search in high-dimensional spaces.



Ying Deng is currently an undergraduate degree with the Computer Science and Technology, Heilongjiang University, China. Her research interests include similarity search in high-dimensional spaces and data mining.